SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Experiments

COURSE NAME: DEEP LEARNING **COURSE CODE: MLA0408**

FACULTY : Dr. D.SUDHAGAR

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 50

Code of Conduct:

I Haresh Kumar N L (192425009) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

QUESTIONS:10 **Total Marks:50**

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

6. Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

7. Forward Propagation and Backpropagation Algorithm

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

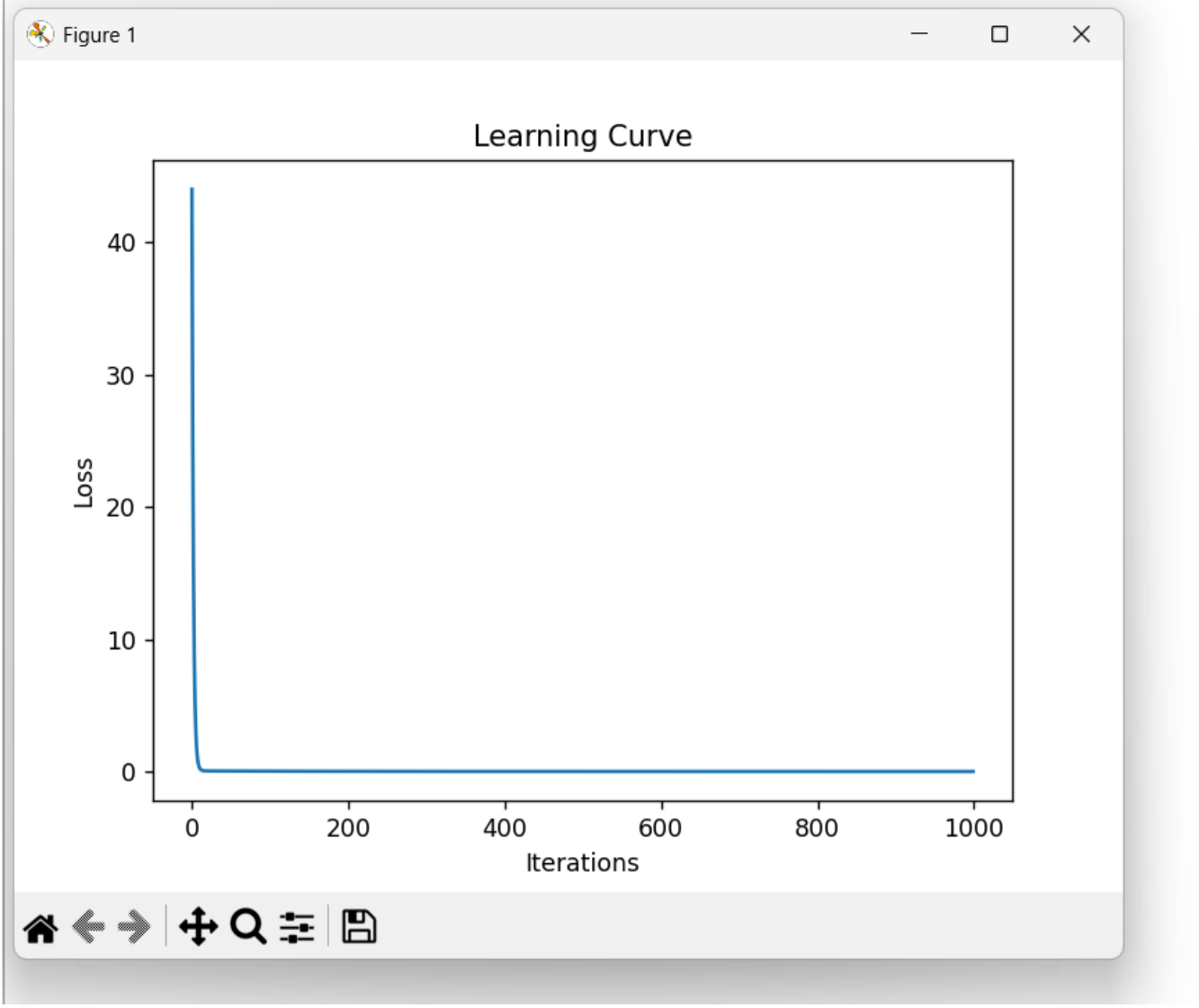
Answer

1) Code:

```
import numpy as np
import matplotlib.pyplot as plt
X = np.array([1,2,3,4,5])
Y = np.array([2,4,6,8,10])
m = 0
c = 0
lr = 0.01
epochs = 1000
n = len(X)
loss = []
for i in range(epochs):
    y_pred = m*X + c
    cost = np.mean((Y-y_pred)**2)
    loss.append(cost)
    dm = (-2/n)*np.sum(X*(Y-y_pred))
    dc = (-2/n)*np.sum(Y-y_pred)
    m -= lr*dm
    c -= lr*dc
print("Slope =",m)
print("Intercept =",c)
plt.plot(loss)
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.title("Learning Curve")
plt.show()
```

Output:

```
==== RESTART: C:/Users/hk838/Downloads/Slot- A,B
Slope = 1.9951803506719779
Intercept = 0.017400463340610635
```



2) Code:

```
import numpy as np
np.random.seed(10)
mu = 5
sigma = 2
data = np.random.normal(mu,sigma,1000)
mu_mle = np.mean(data)
var_mle = np.mean((data-mu_mle)**2)
print("Actual Mean :",mu)
print("Estimated Mean :",mu_mle)
print("Actual Variance :",sigma**2)
print("Estimated Variance :",var_mle)
```

Output:

```
==== RESTART: C:/Users/hk838/Downloads/Slo
Actual Mean : 5
Estimated Mean : 4.970886728769059 5
Actual Variance : 4
Estimated Variance : 3.519059017137644 6
```

3) Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,confusion_matrix
iris = load_iris()
X_train,X_test,Y_train,Y_test = train_test_split(
   iris.data,iris.target,test_size=0.2,random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = LogisticRegression()
model.fit(X_train,Y_train)
pred = model.predict(X_test)
print("Accuracy =",accuracy_score(Y_test,pred))
print(confusion_matrix(Y_test,pred))
```

Output:

```
==== RESTART: C:/U
Accuracy = 1.0
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

4) Code:

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
X,y = make_moons(n_samples=300,noise=0.2,random_state=42)
X_train,X_test,y_train,y_test = train_test_split(
X,y,test_size=0.2)
model = MLPClassifier(hidden_layer_sizes=(10,),
                max_iter=1000)
model.fit(X_train,y_train)
print("Accuracy =",model.score(X_test,y_test))
```

Output:

```
==== RESTART: C:/Users/hk838/I
Accuracy = 0.8666666666666667
```

5) Code:

```
import numpy as np
x = np.array([-2,-1,0,1,2])
sigmoid = 1/(1+np.exp(-x))
relu = np.maximum(0,x)
print("Input :",x)
print("Sigmoid :",sigmoid)
print("ReLU :",relu)
```

Output:

```
==== RESTART: C:/Users/hk838/Downloads/Slot- A,B,C,D/MLA0408/Assess
Input : [-2 -1  0  1  2]
Sigmoid : [0.11920292 0.26894142 0.5        0.73105858 0.88079708]
ReLU : [0 0 0 1 2]
```

6)

a) Code:

```
from sklearn.datasets import make_classification
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
X,y = make_classification(
n_samples=200,
n_features=2,
n_redundant=0,
n_clusters_per_class=1,
random_state=1)
X_train,X_test,y_train,y_test = train_test_split(
X,y,test_size=0.2)
model = Perceptron()
model.fit(X_train,y_train)
print("Accuracy =",model.score(X_test,y_test))
```

Output:

```
==== RESTART: C:/Us
Accuracy = 0.8
```

b) Code:

```
from sklearn.neural_network import MLPClassifier
mlp = MLPClassifier(hidden_layer_sizes=(20,),
            max_iter=1000)
mlp.fit(X_train,y_train)
print("MLP Accuracy =",mlp.score(X_test,y_test))
```

Output:

MLP Accuracy > Perceptron

7)

a) Code:

```
import numpy as np
x = np.array([1,2])
W = np.array([[0.5,0.2],
          [0.3,0.8]])
b = np.array([0.1,0.2])
z = np.dot(x,W)+b
output = 1/(1+np.exp(-z))
print(output)
```

Output:

```
==== RESTART: C:/Users/hk
[0.76852478 0.88079708]
```

b) Code:

```
w = 0.5
lr = 0.1
target = 1
output = 0.8
gradient = (output-target)
new_weight = w-lr*gradient
print("Updated Weight =",new_weight)
```

Output:

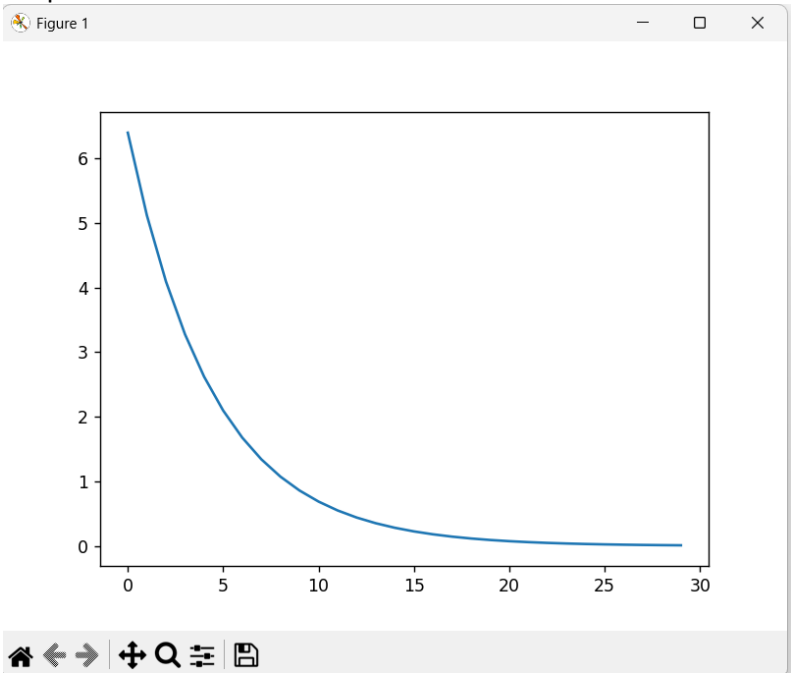
```
==== RESTART: C:/Users/hk83
Updated Weight = 0.52
```

8)

a) Code:

```
import numpy as np
import matplotlib.pyplot as plt
x = 8
lr = 0.1
values=[]
for i in range(30):
    grad = 2*x
    x = x-lr*grad
    values.append(x)
plt.plot(values)
plt.show()
```

Output:



b) Code:

```
from sklearn.linear_model import SGDRegressor
from sklearn.datasets import make_regression
X,y = make_regression(
n_samples=500,
n_features=1,
noise=10)
model = SGDRegressor(max_iter=1000)
model.fit(X,y)
print(model.score(X,y))
```

Output:

```
==== RESTART: C:/Users/h
0.8303796250986655
```

9) Code:

```
from sklearn.neural_network import MLPRegressor
from sklearn.datasets import make_regression
X,y = make_regression(
n_samples=500,
n_features=10)
model = MLPRegressor(
batch_size=32,
max_iter=300)
model.fit(X,y)
print("Training Completed")
```

Output:

```
==== RESTART: C:/Users/hk838/Downloads/Slot-

Warning (from warnings module):
  File "C:\Users\hk838\AppData\Roaming\Pytho
ural_network\_multilayer_perceptron.py", lin
    warnings.warn(
ConvergenceWarning: Stochastic Optimizer: Ma
he optimization hasn't converged yet.
Training Completed
```

10) Code:

```
import numpy as np
from sklearn.metrics import pairwise_distances
for d in [2,10,50,100,500]:
   X = np.random.rand(100,d)
   dist = pairwise_distances(X)
   print("Dimension:",d)
   print("Average Distance:",dist.mean())
```

Output:

```
==== RESTART: C:/Users/hk838/Downloads/Slot
Dimension: 2
Average Distance: 0.522385476003783
Dimension: 10
Average Distance: 1.2932823834279272
Dimension: 50
Average Distance: 2.865633349119977
Dimension: 100
Average Distance: 4.029331220388367
Dimension: 500
Average Distance: 9.013307659270598
```